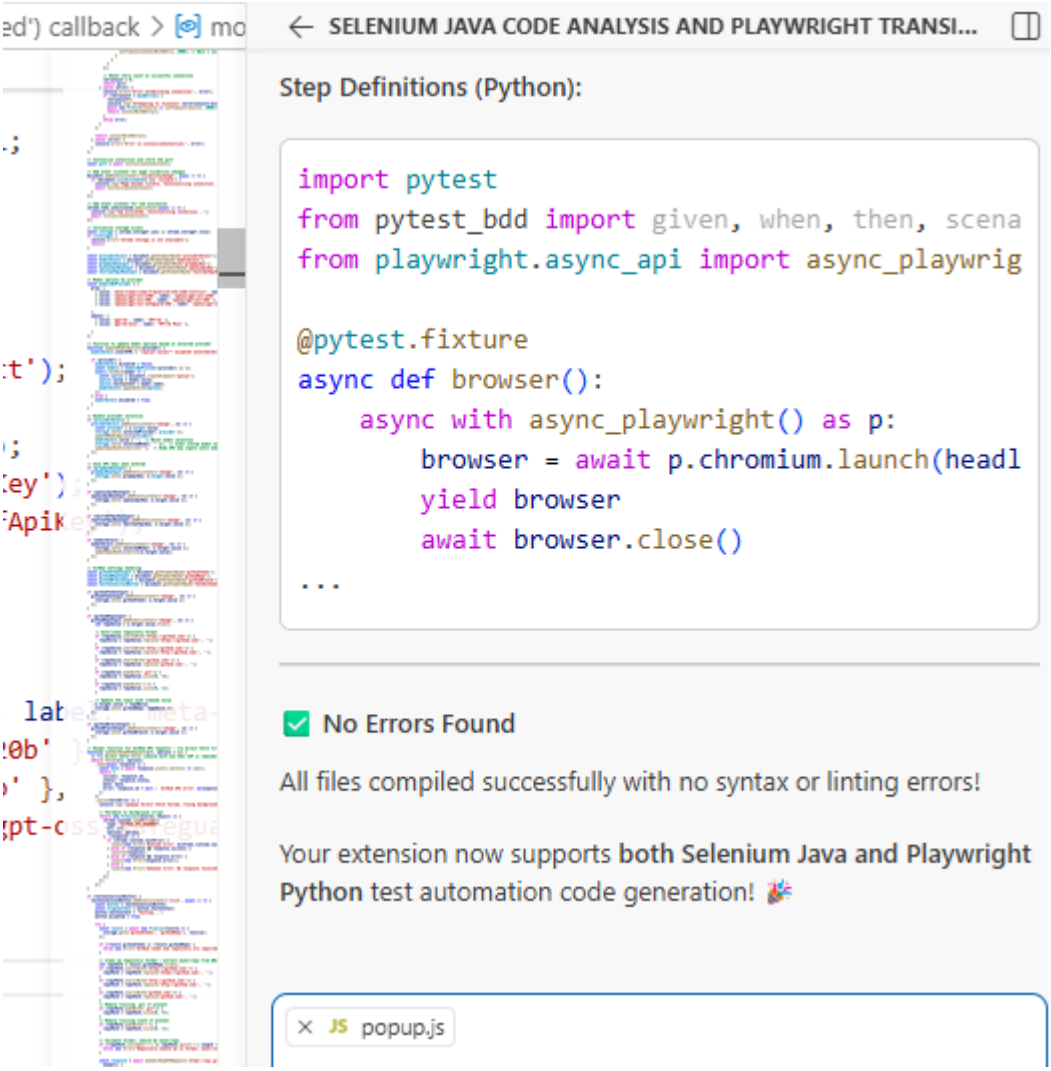
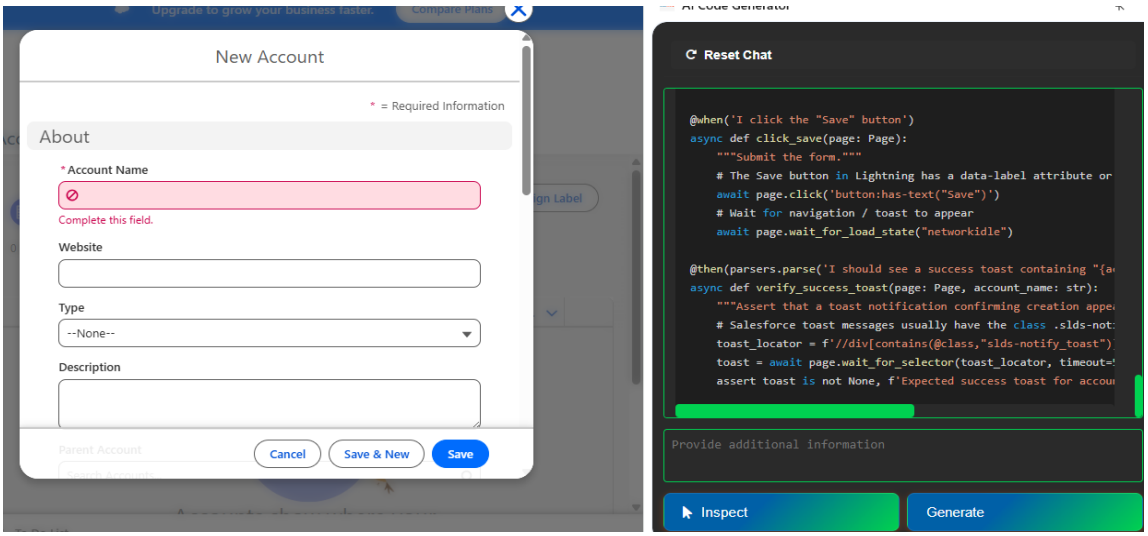
***Week 3- Day 1 Assignment***

Generated Playwright Python code using Copilot:




**Scenario**: Generate Python+Playwright script for New Account creation in Salesforce application.




*Feature*: Create a new Account record in Salesforce Lightning

As a QA engineer,

I want to verify that a user can create an Account with valid data,

So that the system stores the information correctly.

Background:

Given I open the new account page

Scenario Outline: Successful creation of an Account with South-India data

When I fill the "Account Name" field with "<account_name>"

And I fill the "Website" field with "<website>"

And I select "<type>" from the "Type" picklist

And I fill the "Description" field with "<description>"

And I set the "Parent Account" to "<parent_account>"

And I click the "Save" button

Then I should see a success toast containing "<account_name>"

Examples: South India realistic data

| account_name | website | type | description | parent_account |

| Chennai Textiles Ltd | https://ctextiles.in | Customer | Leading textile manufacturer in Chennai. | Global Holdings|

| Mysore Spice Traders | https://mysorespice.com| Partner | Supplier of authentic Mysore spices. | Spice Corp |

| Bangalore Tech Solutions | https://btsolutions.io | Prospect | IT services provider focusing on AI/ML. | Tech Ventures |

```
import pytest

from pytest_bdd import given, when, then, scenarios, parsers

from playwright.async_api import async_playwright, Browser, BrowserContext, Page


# ---------------------------------------------------------------------

# Link the feature file

# ---------------------------------------------------------------------

scenarios("features/create_account.feature")


# ---------------------------------------------------------------------

# Playwright fixtures

# ---------------------------------------------------------------------

@pytest.fixture(scope="session")

async def browser() -> Browser:

    """Launch a Chromium browser for the whole test session."""

    async with async_playwright() as p:
```

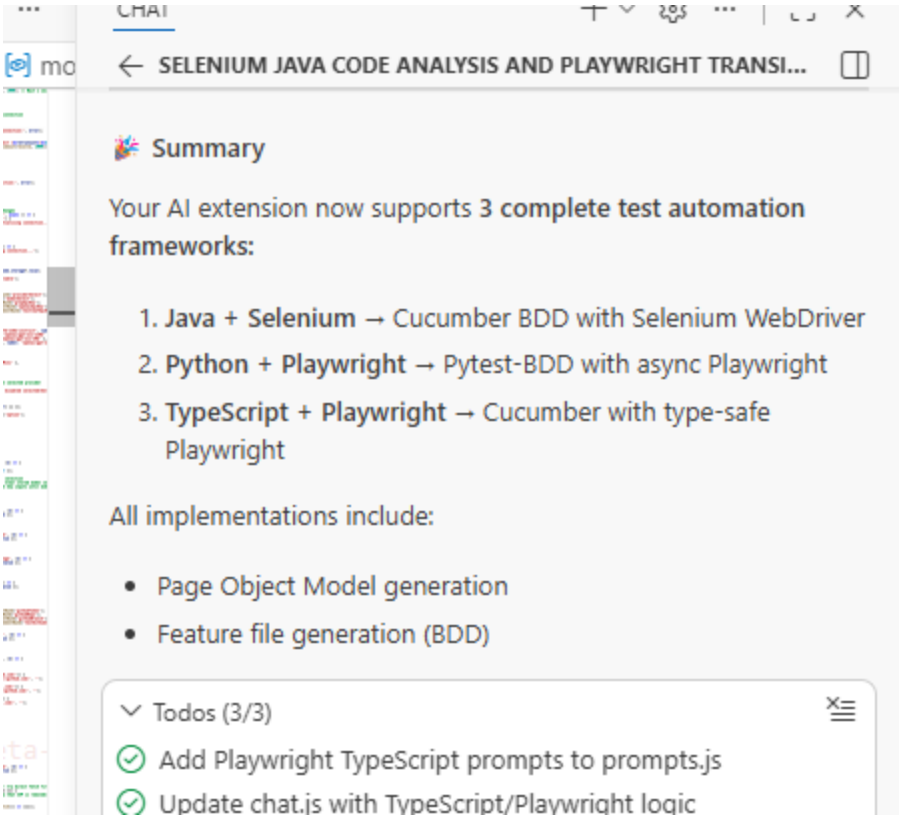
```
    browser = await p.chromium.launch(headless=False)  # set to True in CI
    yield browser
    await browser.close()

@pytest.fixture
async def context(browser: Browser) -> BrowserContext:
  """Create a new isolated browser context for each test."""
  context = await browser.new_context()
  yield context
  await context.close()

@pytest.fixture
async def page(context: BrowserContext) -> Page:
  """Create a new page (tab) for each test."""
  page = await context.new_page()
  yield page
  await page.close()

# ---------------------------------------------------------------------
# Step implementations
# ---------------------------------------------------------------------
@given("I open the new account page")
async def open_new_account_page(page: Page):
  """Navigate directly to the Account creation page."""
  url = (
    "https://nosoftware-speed-9953.lightning.force.com"
```

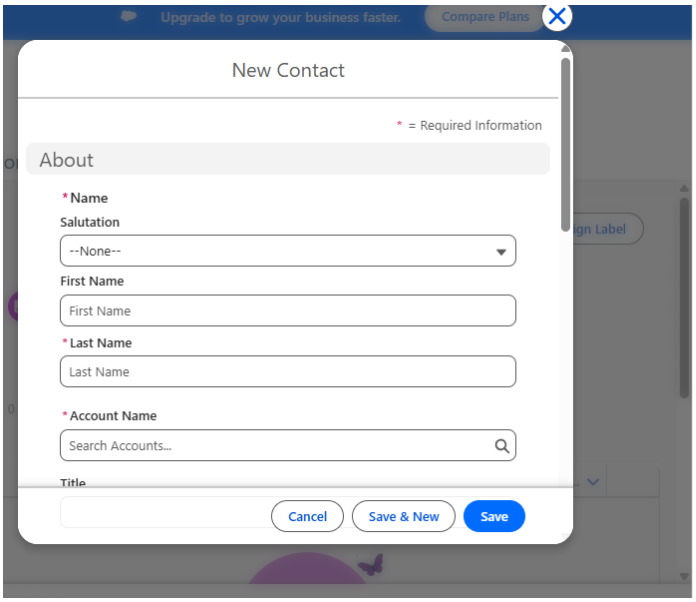
```
        "/lightning/o/Account/new?count=1&nooverride=1&useRecordTypeCheck=1"
        "&navigationLocation=LIST_VIEW&uid=17762558934547303"
"&backgroundContext=%2Flightning%2Fo%2FAccount%2Flist%3FfilterName%3DAllAccounts"
    )
    await page.goto(url)
    # Wait for the form to be ready – the Account Name input is required
    await page.wait_for_selector('input[name="Name"]', state="visible")

@when(parsers.parse('I fill the "Account Name" field with "{account_name}"'))
async def fill_account_name(page: Page, account_name: str):
    """Enter the Account Name."""
    await page.fill('input[name="Name"]', account_name)

@when(parsers.parse('I fill the "Website" field with "{website}"'))
async def fill_website(page: Page, website: str):
    """Enter the Website."""
    await page.fill('input[name="Website"]', website)

@when(parsers.parse('I select "{type}" from the "Type" picklist'))
async def select_type(page: Page, type: str):
    """Open the Type combobox and choose the requested option."""
    # Click the combobox button (identified by its id in the DOM)
    await page.click('#combobox-button-176')
    # The dropdown appears; click the option that matches the text
    # Using a locator that matches the visible text inside the dropdown
```

```
    option_locator = f'//lightning-base-combobox//lightning-base-combobox//div[@role="option"]//span[text()="{type}"]'
    await page.wait_for_selector(option_locator, state="visible")
    await page.click(option_locator)


@when(parsers.parse('I fill the "Description" field with "{description}"'))
async def fill_description(page: Page, description: str):
    """Enter the Description."""
    await page.fill('textarea#input-182', description)


@when(parsers.parse('I set the "Parent Account" to "{parent_account}"'))
async def set_parent_account(page: Page, parent_account: str):
    """Search for a parent account and select the first matching result."""
    # Click into the lookup input
    await page.click('input#combobox-input-240')
    # Type the parent account name
    await page.fill('input#combobox-input-240', parent_account)
    # Wait for the dropdown list to populate and select the first entry
    dropdown_option = '//div[@id="dropdown-element-240"]//span[contains(@class,"slds-truncate")]'
    await page.wait_for_selector(dropdown_option, state="visible")
    # Click the first matching option (exact match if possible)
    await page.click(f'{dropdown_option}[text()="{parent_account}"]')


@when('I click the "Save" button')
async def click_save(page: Page):
    """Submit the form."""
```

```
    # The Save button in Lightning has a data-label attribute or visible text
    await page.click('button:has-text("Save")')
    # Wait for navigation / toast to appear
    await page.wait_for_load_state("networkidle")


@then(parsers.parse('I should see a success toast containing "{account_name}"'))
async def verify_success_toast(page: Page, account_name: str):
    """Assert that a toast notification confirming creation appears."""
    # Salesforce toast messages usually have the class .slds-notify_toast
    toast_locator = f'//div[contains(@class,"slds-notify_toast")]//span[contains(text(),"{account_name}")]'
    toast = await page.wait_for_selector(toast_locator, timeout=5000)
    assert toast is not None, f'Expected success toast for account "{account_name}" not found.'
```

Playwright+Typescript code implementation is done using Copilot:




***Playwright + Type script code Generation:***